

Федеральное государственное автономное образовательное учреждение высшего образования «Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерных технологий

Теория функций комплексной переменной

Лабораторная работа №1

Выполнила:

Павличенко Софья Алексеевна, Р3215

Преподаватель:

Бойцев Антон Александрович

Санкт-Петербург 2024г.

Оглавление

Задание.....	3
Решение	4
Задание 1	4
Свойство 1	4
Свойство 2.....	5
Задание 2	6
Исходный код программы	6
Полученные визуализации	7
Задание 3	9
Исходный код программы	9
Полученные визуализации	10
Задание 4.....	13
Структура фрактала	13
Построение фрактала	13
Исходный код программы	14
Полученные визуализации	15
Заключение.....	17

Задание

1. Докажите свойства 1 и 2 для множества Мандельброта.
2. Напишите программу, которая будет строить визуализацию множества Мандельброта. Выберите разумные ограничения, поварьируйте максимальное количество итераций. Попробуйте приблизить отдельные части множества, чтобы увидеть фрактальную структуру.
3. Напишите программу, которая по заданному c строит заполненное множество Жюлиа. Поварьируйте максимальное количество итераций, попробуйте пронаблюдать фрактальную структуру, рассмотрите множество при разных c . (Например, красиво получается при $c = -0.5251993 + i0.5251993$).
4. Найдите какой-нибудь неразобранный фрактал (например, бассейны Ньютона). Опишите его структуру, построение. Нарисуйте визуализации. Будьте готовы выступить с докладом перед своими одноклассниками.

Решение

Задание 1

Свойство 1

Множество Мандельброта переходит само в себя при сопряжении. Иными словами, оно симметрично относительно вещественной оси.

□

Модуль (длина) комплексного числа не изменяется при сопряжении:

$$|z_n| = |\overline{z_n}|$$

Поэтому, если $|z_n|$ ограничено (то есть, существует R такое, что $|z_n| \leq R$ для всех n), то:

$$|z_n| = |\overline{z_n}| \leq R,$$

что показывает, что $\overline{z_n}$ также ограничено.

То есть ограниченность или неограниченность орбиты не изменится, если каждый её элемент подвергнуть комплексному сопряжению.

Осталось доказать, что орбиты нуля для z и $\bar{z}$ симметричны друг другу при отражении относительно вещественной оси. Для z получаем орбиту:

$$O(z) = \langle 0, z, z + z^2, z + z^2 + 2z^3 + z^4, \dots \rangle,$$

а для $\bar{z}$ —

$$O(\bar{z}) = \langle 0, \bar{z}, \bar{z} + \overline{z^2}, \bar{z} + \overline{z^2} + 2\overline{z^3} + \overline{z^4}, \dots \rangle$$

Поскольку все элементы орбиты $O(z)$ являются многочленами от z с вещественными коэффициентами, то комплексное сопряжение можно вынести за знак многочленов: $g(\bar{z}) = \overline{g(z)}$. Итак, $O(\bar{z}) = \overline{O(z)}$, что и требовалось доказать.

■

Свойство 2

Если $|c| > 2$, то c не принадлежит множеству Мандельброта.

□

Для доказательства покажем, что последовательность z_n стремится к бесконечности, если $|c| > 2$, с помощью метода математической индукции.

База индукции ($n = 0$)

$z_0 = 0$, и, поскольку $|c| > 2$, имеем: $|z_1| = |z_0^2 + c| = |c| > 2$. Таким образом, база индукции выполняется.

Шаг индукции

Пусть для некоторого $n \geq 1$ выполняется $|z_n| > 2$. Докажем, что тогда $|z_{n+1}| > 2$.

Рассмотрим:

$$|z_{n+1}| = |z_n^2 + c|.$$

Согласно треугольному неравенству:

$$|z_{n+1}| = |z_n^2 + c| \geq |z_n|^2 - |c|.$$

Поскольку $|z_n| > 2$, имеем:

$$|z_n|^2 > 4.$$

А так как $|c| > 2$, то $|z_n|^2 - |c| > 4 - 2 = 2$.

Следовательно, $|z_{n+1}| > 2$.

Это завершает шаг индукции, и, следовательно, по принципу математической индукции для всех n выполняется $|z_n| > 2$.

Из полученного следует, что если $|c| > 2$, то последовательность z_n неограниченна, что означает, что c не принадлежит множеству Мандельброта.

■

Задание 2

Исходный код программы

```
import pygame
import os

WHITE = (255, 255, 255)
BLACK = (0, 0, 0)

x = 400
y = 100
os.environ['SDL_VIDEO_WINDOW_POS'] = "%d,%d" % (x,y)

pygame.init()

W = 700
H = 700

sc = pygame.display.set_mode((W, H))
pygame.display.set_caption("Множества Мандельброта")
sc.fill(WHITE)

clock = pygame.time.Clock()

P = 350                                     # размер [2*P+1 x 2*P+1]
scale = P / 2                             # масштабный коэффициент
n_iter = 100                               # число итераций для проверки принадлежности к
множеству Жюлиа
view = (0, 0)

for x in range(-P + view[0], P + view[0]):
    for y in range(-P + view[1], P + view[1]):
        a = x / scale
        b = y / scale
        c = complex(a, b)
        z = complex(0)
        n = 0
        for n in range(n_iter):
            z = z**2 + c
            if abs(z) > 2:
                break

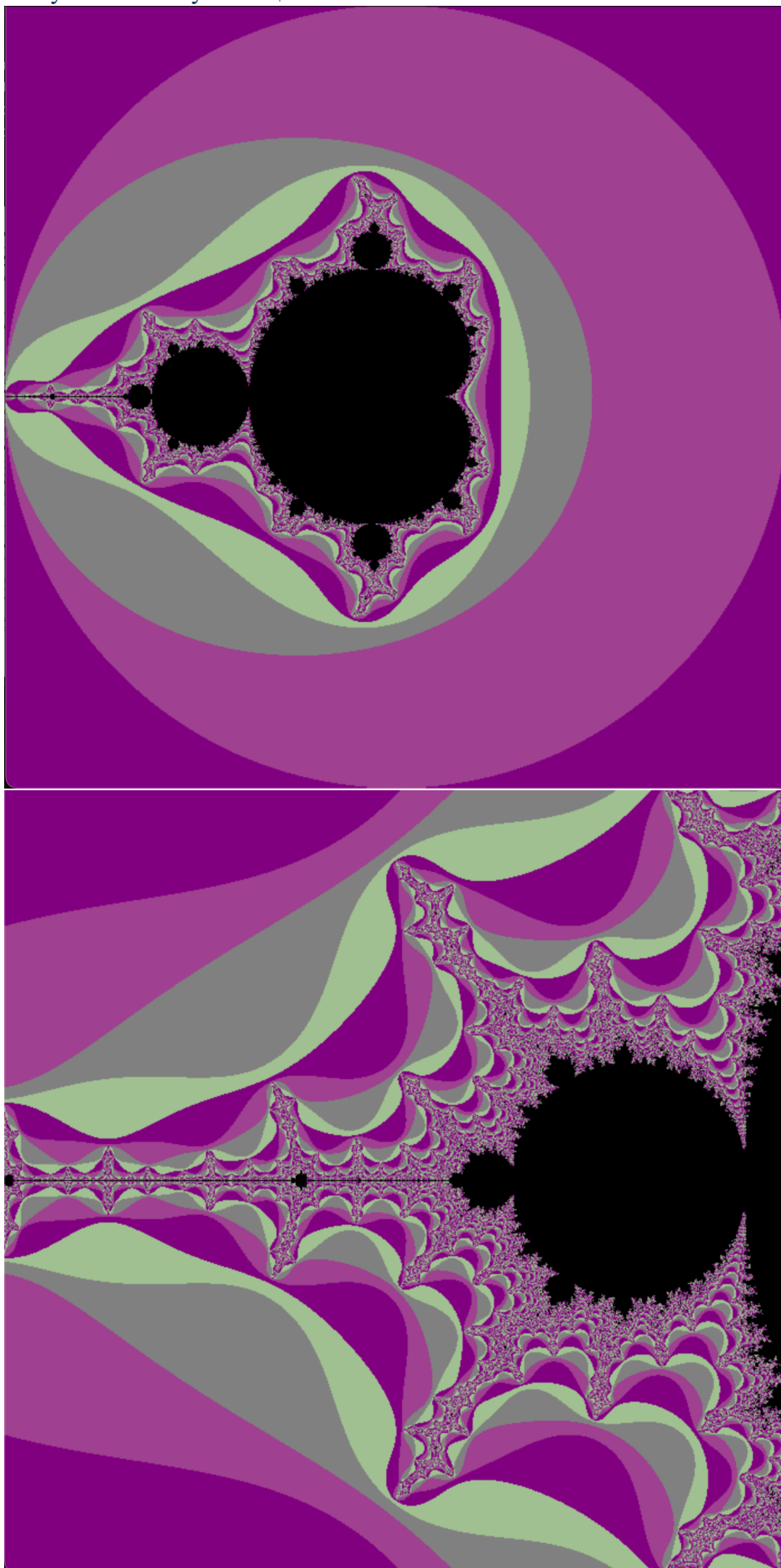
        if n == n_iter-1:
            r = g = b = 0
        else:
            r = (n % 2) * 32 + 128
            g = (n % 4) * 64
            b = (n % 2) * 16 + 128

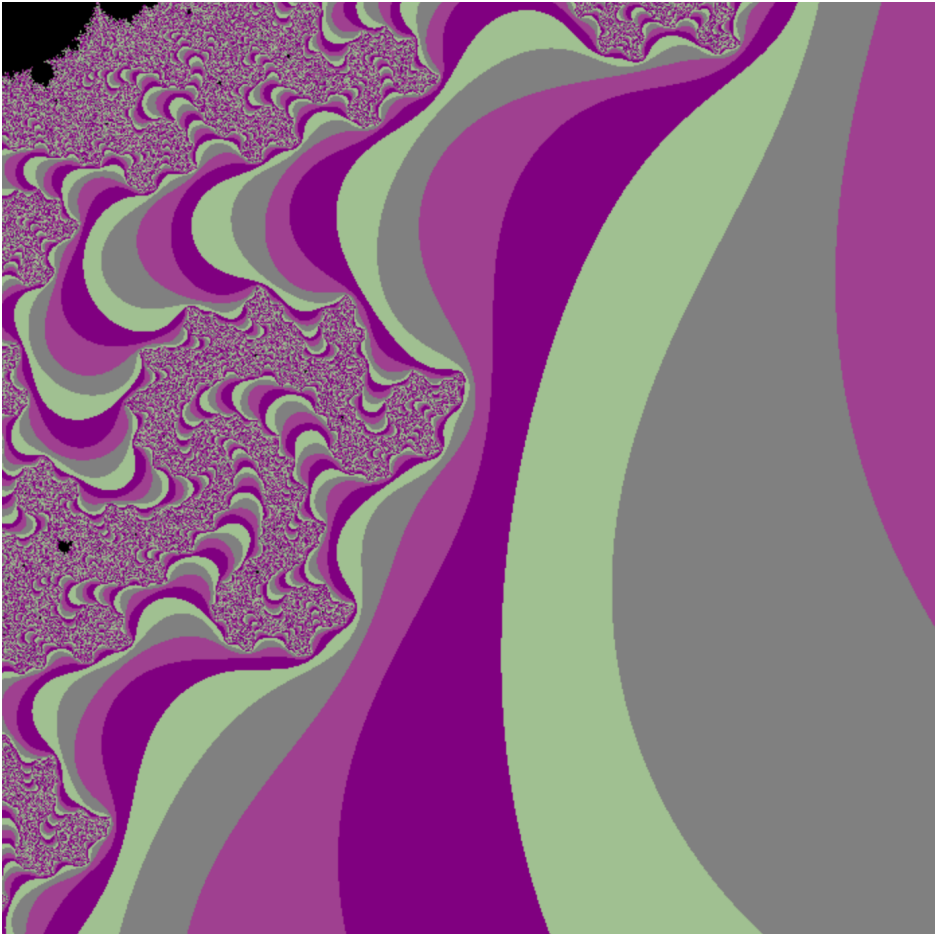
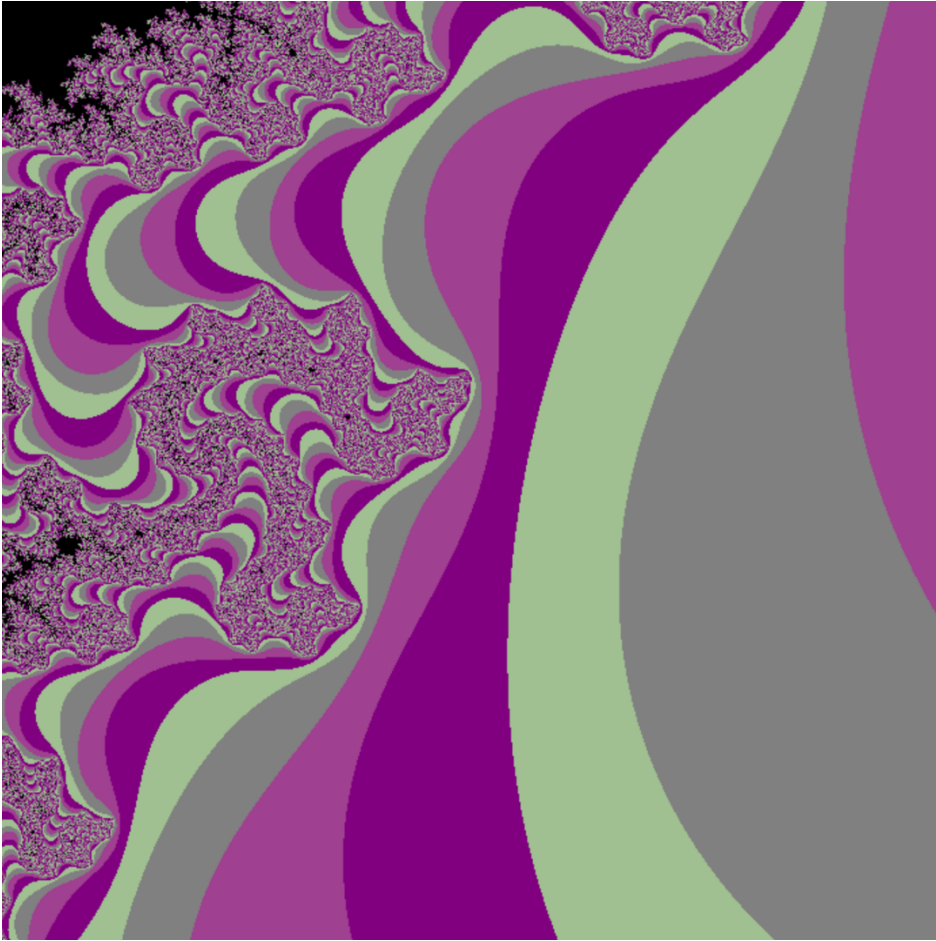
        pygame.draw.circle(sc, (r, g, b), (x + P - view[0], y + P - view[1]),
1)

while True:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            exit()

    pygame.display.update()
    clock.tick(30)
```

Полученные визуализации





Задание 3

Исходный код программы

```
import pygame
import os

WHITE = (255, 255, 255)
BLACK = (0, 0, 0)

x = 400
y = 100
os.environ['SDL_VIDEO_WINDOW_POS'] = "%d,%d" % (x, y)

pygame.init()

W = 700
H = 700

sc = pygame.display.set_mode((W, H))
pygame.display.set_caption("Множества Жюлиа")
sc.fill(WHITE)

clock = pygame.time.Clock()

# c = complex(-0.5251993, 0.5251993)
# c = complex(-1)
# c = complex(0.377, -0.248)
c = complex(-0.2, 0.75)
P = 350 # размер [2*P+1 x 2*P+1]
scale = P / 2 # масштабный коэффициент
n_iter = 100 # число итераций для проверки принадлежности к множеству Жюлиа
view = (0, 0)

for x in range(-P + view[0], P + view[0]):
    for y in range(-P + view[1], P + view[1]):
        a = x / scale
        b = y / scale
        z = complex(a, b)
        n = 0
        for n in range(n_iter):
            z = z**2 + c
            if abs(z) > 2:
                break

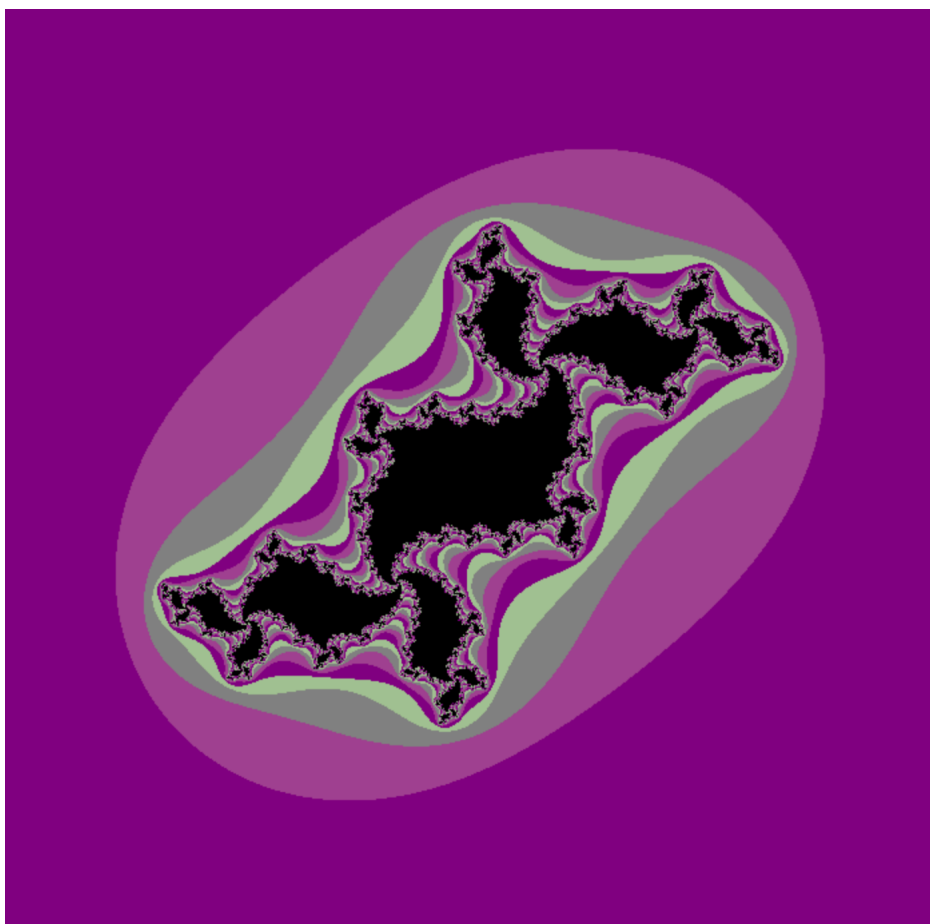
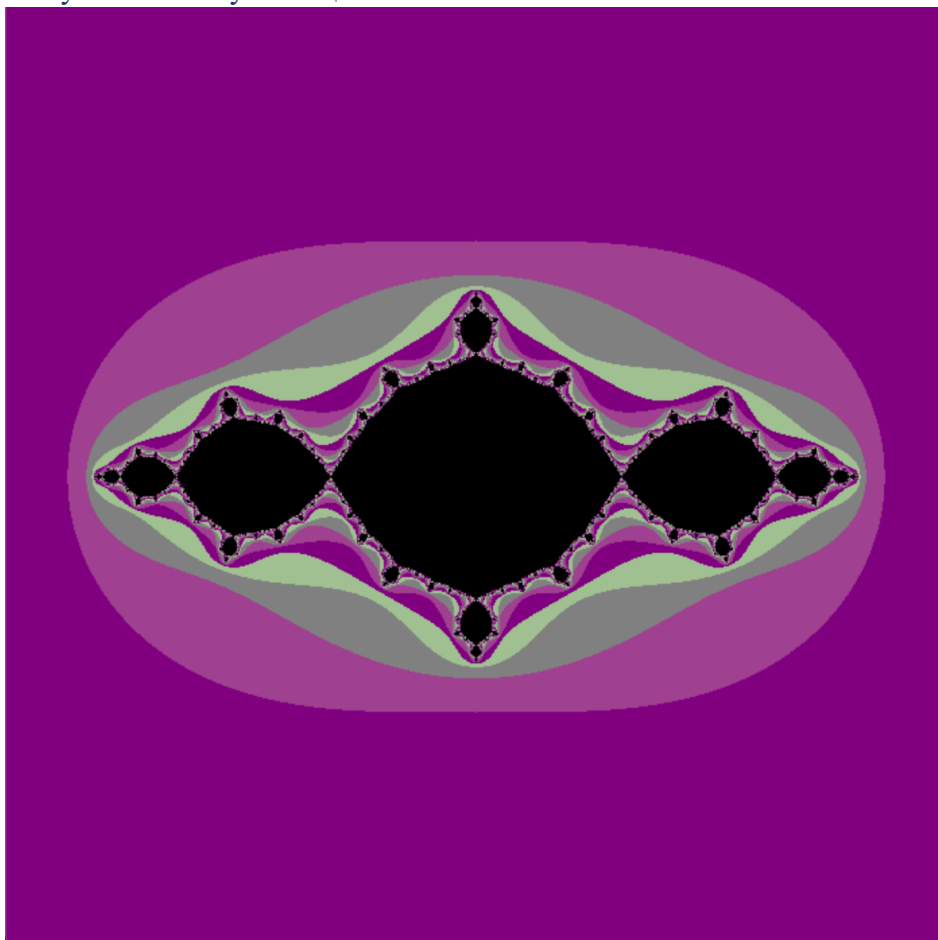
        if n == n_iter-1:
            r = g = b = 0
        else:
            r = (n % 2) * 32 + 128
            g = (n % 4) * 64
            b = (n % 2) * 16 + 128

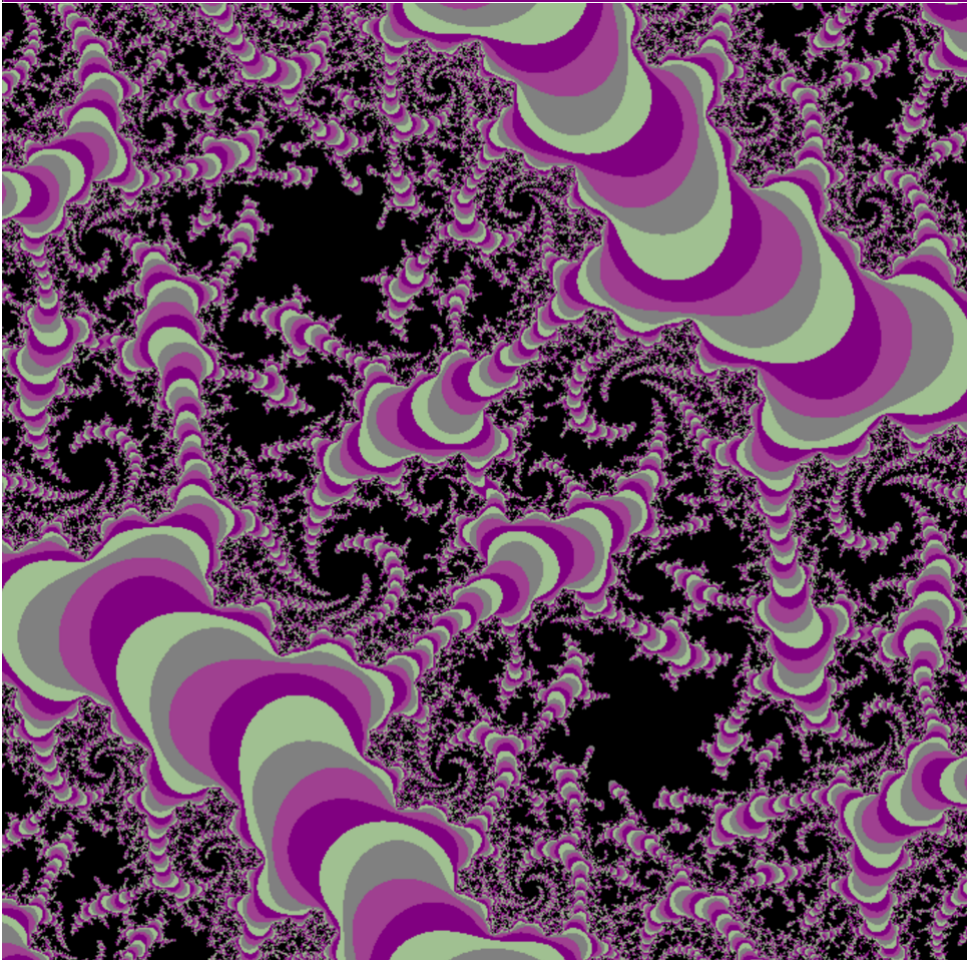
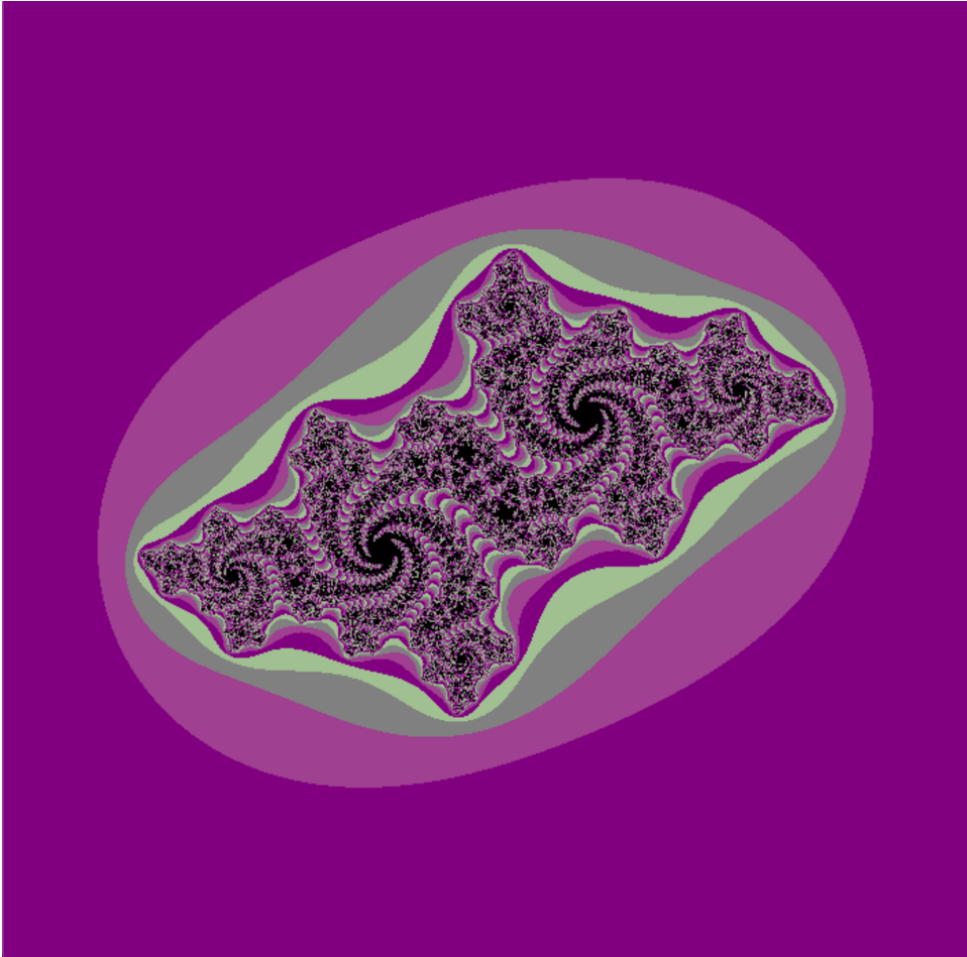
        pygame.draw.circle(sc, (r, g, b), (x + P - view[0], y + P - view[1]),
1)

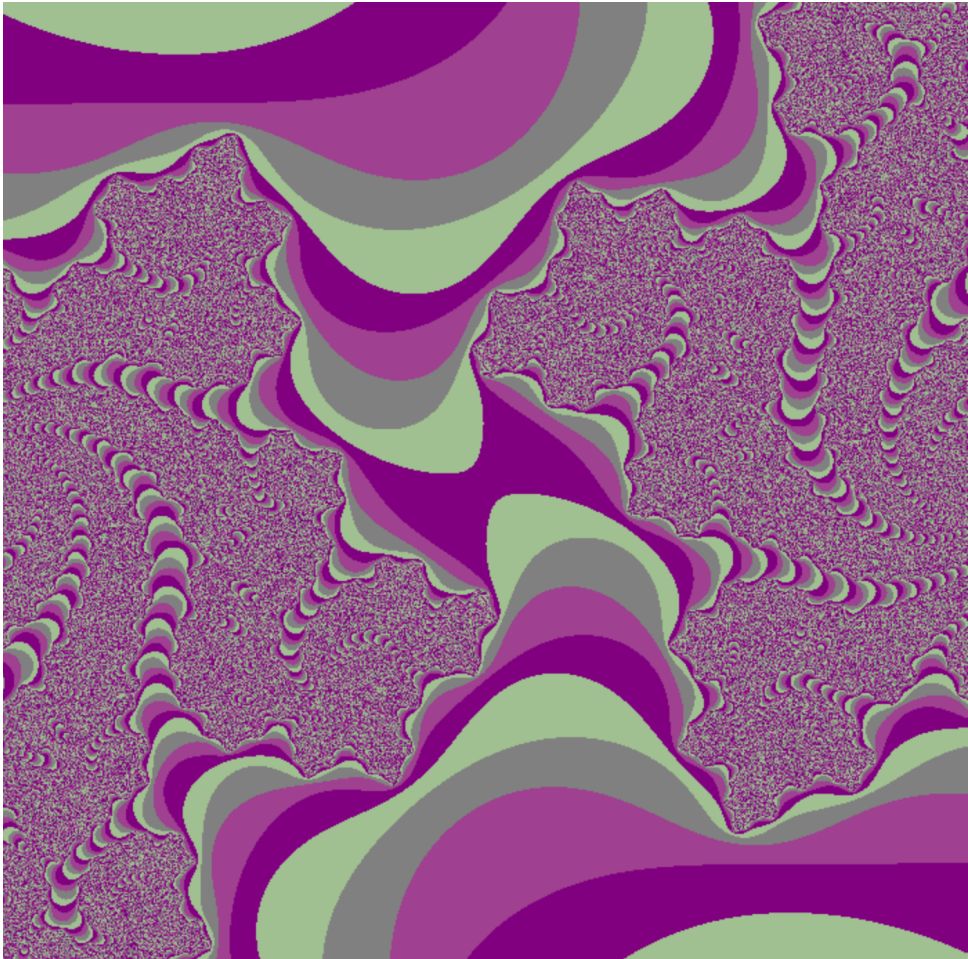
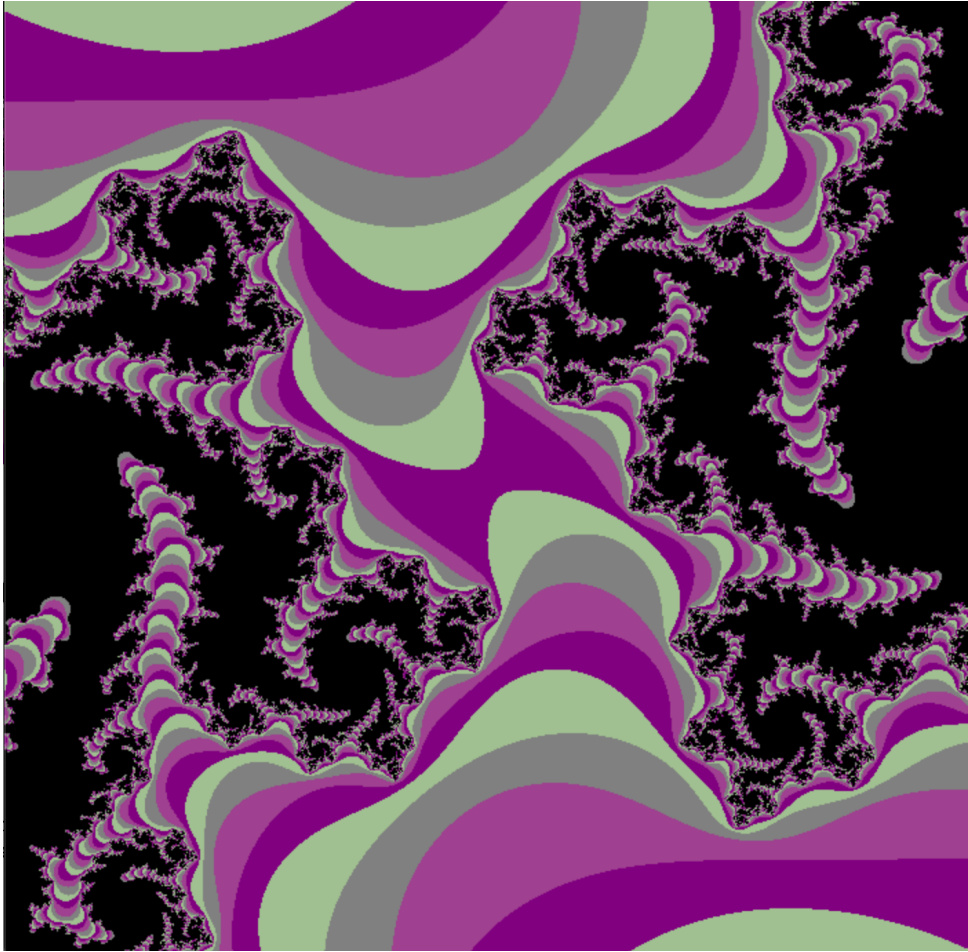
while True:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            exit()

    pygame.display.update()
    clock.tick(30)
```

Полученные визуализации







Задание 4

Фрактал бассейна Ньютона – это графическое представление процесса нахождения корней комплексных функций методом Ньютона. Он иллюстрирует, к какому из нескольких корней стремится каждое начальное значение, если итеративно применять метод Ньютона.

Структура фрактала

Для уравнения с тремя корнями, как $z^3 = 1$, существует три корня: 1 , ω , и ω^2 , где $\omega = e^{2\pi i/3}$ — это комплексные корни кубического уравнения. На комплексной плоскости каждая точка стремится к одному из этих корней при последовательных итерациях. Визуально фрактал бассейна Ньютона делится на крупные области, каждая из которых привязана к своему корню. Все точки внутри такой области сходятся к одному и тому же корню, создавая однородные по цвету зоны.

Однако по мере приближения к границам между этими областями картина становится сложнее: здесь области разных цветов начинают перемешиваться, образуя бесконечно детализированные узоры, которые повторяются на всё меньших масштабах. Эти фрактальные границы отражают высокую чувствительность метода Ньютона к начальным условиям, когда даже небольшое изменение приводит к смене конечного корня, к которому сходятся итерации.

Построение фрактала

Для построения фрактала бассейна Ньютона используется итерационная формула метода Ньютона:

$$z_{n+1} = z_n - \frac{f(z_n)}{f'(z_n)}$$

Для уравнения $z^3 = 1$, функция будет:

$$f(z) = z^3 - 1$$

Её производная:

$$f'(z) = 3z^2$$

Тогда итерационная формула примет вид:

$$z_{n+1} = z_n - \frac{z_n^3 - 1}{3z_n^2} = \frac{2z_n + \frac{1}{z_n^2}}{3}$$

Исходный код программы

```
import pygame
import os

WHITE = (255, 255, 255)

x = 400
y = 100
os.environ['SDL_VIDEO_WINDOW_POS'] = "%d,%d" % (x, y)

pygame.init()

W = 700
H = 700

sc = pygame.display.set_mode((W, H))
pygame.display.set_caption("Бассейны Ньютона")
sc.fill(WHITE)

clock = pygame.time.Clock()

max_value = 1e+6
min_value = 1e-6
P = 350
scale = 0.005 # масштабирование для преобразования координат
n_iter = 50
view = (0, 0)

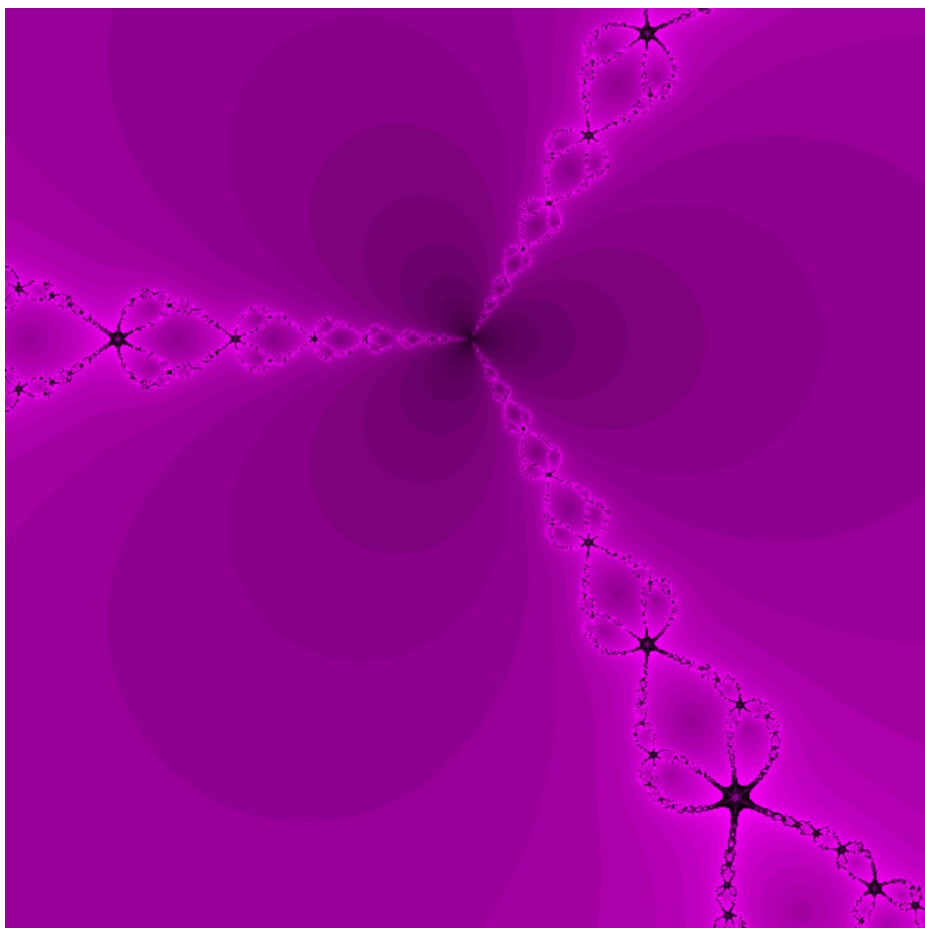
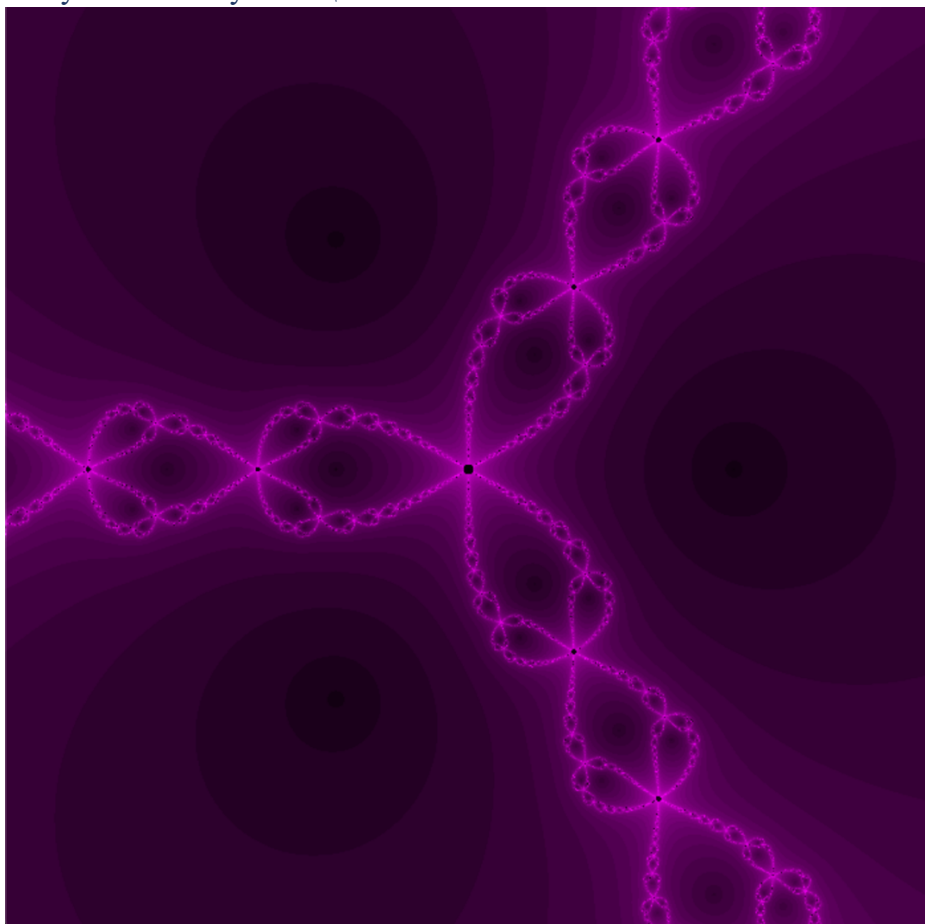
# Генерация бассейнов Ньютона
for x in range(-P + view[0], P + view[0]):
    for y in range(-P + view[1], P + view[1]):
        n = 0
        z_x = x * scale
        z_y = y * scale
        d_x, d_y = z_x, z_y # d - разница между z и предыдущим z
        while (z_x**2 + z_y**2 < max_value) and (d_x**2 + d_y**2 > min_value)
and (n < n_iter):
            t_x, t_y = z_x, z_y
            # Метод Ньютона для функции  $z^3 - 1$ 
            p = (t_x**2 + t_y**2)**2
            z_x = (2/3) * t_x + (t_x**2 - t_y**2) / (3 * p)
            z_y = (2/3) * t_y * (1 - t_x / p)
            d_x = abs(t_x - z_x)
            d_y = abs(t_y - z_y)
            n += 1

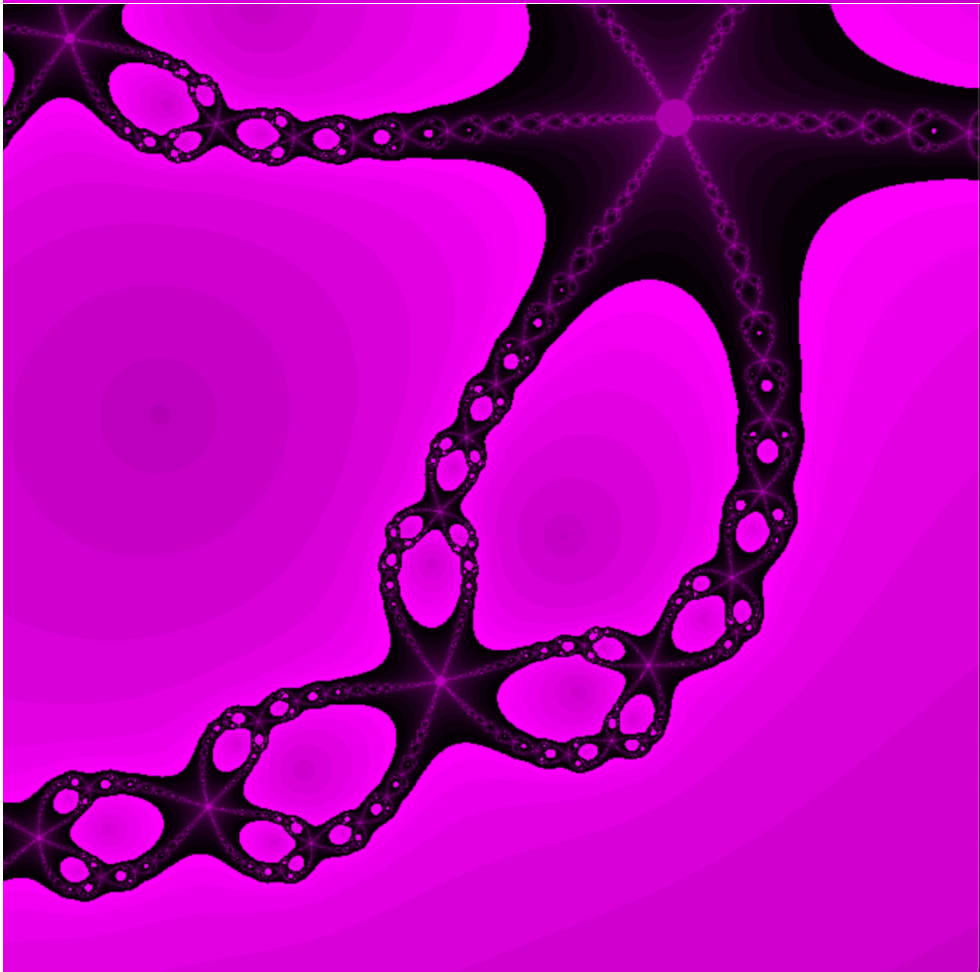
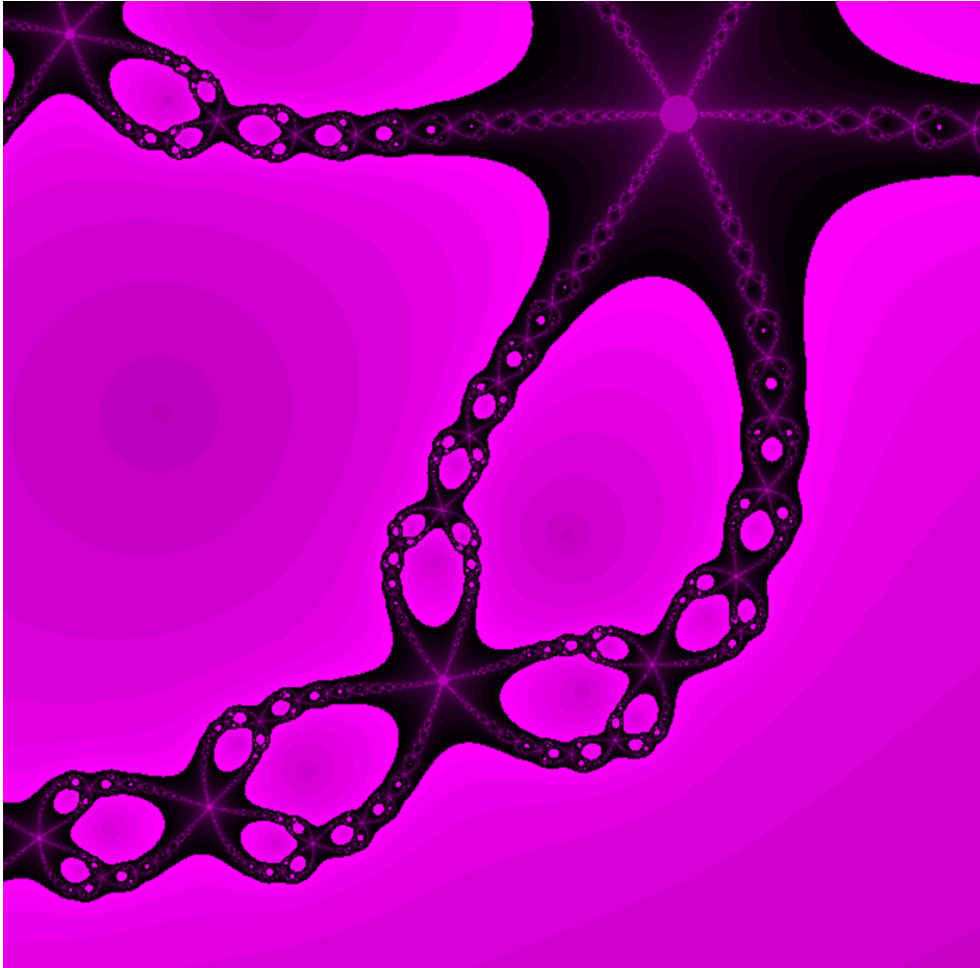
        col = (n * 9) % 255
        pygame.draw.circle(sc, (col, 0, col), (x + P - view[0], y + P -
view[1]), 1)

while True:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            exit()

    pygame.display.update()
    clock.tick(30)
```

Полученные визуализации





Заключение

Эта лабораторная работа дала практический опыт работы с фракталами, от построения и визуализации множества Мандельброта и Жюлиа до изучения более сложных фракталов, таких как бассейны Ньютона. Каждый из них продемонстрировал уникальные свойства, которые помогают понять природу фрактальных структур и их важность в математическом моделировании.